

Discrete Mathematics for BSAI

BS Artificial Intelligence · Semester 1st

Instructor/Author: Dr. Hira Benish

Programming environment: Google Colab

PMAI Lab

Course design

This planner assumes no previous discrete mathematics, proof writing, or Python. A mathematical idea begins with a familiar situation, then moves through intuition, notation, visual models, worked scenarios, short Colab explorations where useful, and reflection. Python is a supporting tool; all essential mathematics is first done and explained without code. A week has 180 contact minutes. The suggested allocations include teaching, guided practice, and assessment; independent practice is completed outside class.

Four major semester case studies:

CS1 in Week 2, *Can the System Conclude This?*;

CS2 in Week 6, *Organizing an Intelligent Learning System*;

CS3 in Weeks 11–12, *Exploring Possible Feature Sets*;

CS4 in Weeks 13–14, *Campus Inspection and Delivery Robot*.

Smaller application case: CS5 in Week 15, *Exploring a Decision/Search Tree*.

A case study is developed as a complete worked example in the eventual learning module; the planner identifies its mathematical model and classroom task.

16-Week Planner

Week	Topics Covered	Learning Outcomes	AI/Real-System Connection	Python/Google Colab Integration	Activities & Assessments
1	Mathematical reasoning and propositional logic: everyday decisions, statements, truth values, negation, conjunction, disjunction, implication, and basic truth tables.	Distinguish propositions from questions; translate short device rules into symbols; evaluate a two-variable truth table.	A smart device uses rules such as “if motion is detected, then turn on the light”; distinguish the rule from its converse.	First Colab cell: run True, False, print(), and, or, not; explain each token before use.	Logic-card game; pairs build and swap truth tables; short, low-stakes exit check.
2	Predicate logic and rules of inference: predicates, quantifiers over stated finite domains, negation of $\forall$ and $\exists$, modus ponens, modus tollens; valid versus invalid inference.	Write and interpret quantified statements; test whether a conclusion follows from explicit rules and facts.	Major CS1 — Can the System Conclude This? A rule-based student-support decision system. Given attendance/assessment facts and transparent rules, decide which support recommendation logically follows; discuss missing information.	Explain variables, if/else, and comparison on a tiny Boolean rule; no unexplained data structures.	Decision cards; inference sorting; complete a short case record with rule, fact, conclusion, and limitation.
3	Mathematical reasoning and proof techniques: argument structure; direct proof of an implication; logical equivalence, including $P \Rightarrow Q \equiv \neg Q \Rightarrow \neg P$; proof by contraposition; proof by contradiction.	State assumptions and conclusions; verify equivalence of an implication and its contrapositive; choose and write a short parity/divisibility proof; find counterexamples.	Authentication rule: distinguish proof that a stated policy follows from its assumptions from a few successful tests; parity offers cleaner proof examples.	Optional truth-table check of $P \Rightarrow Q$ and its contrapositive using Week 1 constructs; written proof remains the assessment.	Practice: proof-strip ordering and counterexample hunt. Formal: brief proof assignment with feedback.
4	Proof by mathematical induction: base case, inductive hypothesis, inductive step; simple sums and a recursively growing pattern.	Explain why checking examples is insufficient; complete a basic induction proof and identify an incorrect inductive step.	Verify a claim about the number of items added to a growing digital pattern; use a sum proof without forcing an AI setting.	Optional short table of the first few cases using print(); no new syntax required.	Domino demonstration; fill and critique an induction proof; one formative quiz.
5	Set theory; functions and mappings: membership, subsets, set operations, Cartesian products; domain/codomain/range; introductory one-to-one and onto mappings; simple composition and inverse functions.	Read set notation and Venn diagrams; form small Cartesian products; distinguish a function from an arbitrary assignment; compute a simple composition and recognize when an inverse exists on a specified domain.	Filter a small student or device dataset using set operations; map an input category to a label and explain when mapping back is impossible.	Introduce Python set literals and union/intersection/difference by comparing outputs to hand-drawn Venn diagrams; introduce a one-line def only after function notation is understood.	Venn sorting; arrow-diagram mapping cards; composition relay and exit check.
6	Relations: ordered pairs and properties; equivalence relations and partitions; partial orderings and simple Hasse diagrams.	Test reflexivity, symmetry, transitivity, and antisymmetry on small examples; form equivalence classes; recognize a prerequisite relation as a partial order.	Major CS2 — Organizing an Intelligent Learning System. Within one system, group learners by the same stated support category (an equivalence relation and partition), then order learning modules by prerequisite (a partial order). Compare what each relation represents.	Represent four or five ordered pairs as a set of tuples; inspect membership. Hasse diagram is drawn manually.	Relation-card sorting; support-category grouping activity; draw the case study’s grouping and Hasse diagram; short assessment.

Week	Topics Covered	Learning Outcomes	AI/Real-System Connection	Python/Google Colab Integration	Activities & Assessments
7	Introductory Number Theory: divisibility, primes, greatest common divisor, Euclidean algorithm, modular arithmetic; midterm synthesis.	Compute a small GCD and remainder; interpret a congruence; connect logic, sets, relations, and proofs in short review questions.	Repeating shift or appointment schedules motivate remainders; a small authentication/check-code illustration uses modular arithmetic only when the underlying rule is clear.	Show % on a few integers; optionally verify a hand-computed GCD using a clearly explained built-in/library call.	Clock-face remainder game; Euclidean algorithm pairs; ungraded midterm diagnostic and feedback.
8	Midterm examination				Midterm examination.
9	Discrete sequences and series: indexed terms, arithmetic and geometric patterns, finite sums and sigma notation; distinguish sequence from sum	Express a sequence with a term rule; calculate a finite series; interpret indexing and finite summation without calculus.	Weekly device readings or message counts: separate the value on day n from the total through day n .	Introduce lists, zero-based indexing, for loops, and sum() one at a time to tabulate and compare a short sequence.	Pattern-discovery cards; plot a short sequence by hand and in Colab; reading-count challenge.
10	Recurrence relations and recursive functions: initial conditions, one-step recurrences, repeated substitution, recursion versus iteration; simple growth models	Compute terms from a recurrence and its initial value; compare a recurrence with an explicit rule where simple; trace a small recursive definition.	A queue/backlog or savings balance changes from one step to the next; explain what the model assumes.	Reuse lists and loops to generate terms; then introduce a short def returning a recursive value and manually trace a small input. Avoid efficiency claims about large inputs.	Pass-the-value recurrence game; term-table visualization; mini notebook with a sentence explaining the pattern.
11	Counting foundations and permutations I — DOES ORDER MATTER? Addition/product rules; inclusion–exclusion for two sets; pigeonhole principle; ordering distinct objects.	Count outcomes by cases and stages; correct a double count; use a small pigeonhole argument; count ordered arrangements of distinct objects.	Major CS3 begins — Exploring Possible Feature Sets. With eight understandable candidate features, contrast “select 3” with “rank the top 3”; reason about possible pairs without claiming to evaluate prediction quality.	Enumerate tiny outcomes with loops already learned; compare hand counts to a short printed list. No library imports needed.	Practice: menu-choice game, overlapping-set puzzle, pigeonhole challenge and card arrangement. Formal: short counting-method quiz; distribute CS3 case data.
12	Permutations II and combinations — DOES ORDER MATTER? Restricted arrangements and selection without order; ${}_nP_r$, $C(n, r)$, factorial, and the choice-versus-order decision.	Decide and justify whether a problem calls for a sum/product rule, permutation, combination, inclusion–exclusion, or pigeonhole reasoning; calculate small counts without relying on formula recall alone.	Major CS3 concludes — Exploring Possible Feature Sets. Count two-feature subsets; compare ordered pairs; impose a required feature; exclude pairs containing two incompatible features. Explain why counting possibilities does not identify good predictive features.	Explain factorial by expansion; use a small printed enumeration to verify a hand solution. Optional, explained itertools example follows the mathematics.	Practice: selection-versus-ranking game and peer explanations. Formal: CS3 case submission with justified counts and overcounting check.
13	Elements of graph theory and Euler graphs: vertices/edges, degree, walk/trail/path/cycle, adjacency representations, Euler trail/circuit and degree test for connected graphs.	Model a small route network as a graph; compute degrees; determine whether every edge can be traversed exactly once under the taught conditions.	Major CS4 begins — Campus Inspection and Delivery Robot. Buildings are vertices and walkways edges; decide whether inspection of each walkway once is possible and state route assumptions.	Introduce a small adjacency dictionary after a drawn graph; read neighbors and degree. Explain key/value pairs before use.	Human graph construction with cards/string; Euler route challenge; team route sketch and explanation.

Week	Topics Covered	Learning Outcomes	AI/Real-System Connection	Python/Google Colab Integration	Activities & Assessments
14	Elementary planar graphs, graph coloring, and Hamiltonian paths/cycles: crossing-free drawings; Euler's formula for simple connected planar drawings; proper vertex coloring; visiting every vertex once (path/cycle) versus every edge once	Check $V - E + F = 2$ on a simple connected planar drawing; color a small conflict graph using available slots; distinguish Euler routes from Hamiltonian paths/cycles, without treating the Euler degree test as a Hamiltonian test.	Major CS4 concludes — Campus Inspection and Delivery Robot: compare inspecting each walkway with visiting each building once. For the separate coloring activity, examinations/tasks are vertices, conflicts are edges, and available time slots/resources are colors.	Optional hand-first coloring experiment on a four-to-six-vertex graph; trace an adjacency list manually before running a tiny checker.	Practice: crossing/redrawing activity, a substantial time-slot coloring game, and route comparison. Formal: CS4 route explanation/submission.
15	Rooted trees and traversals: root, parent/child, leaf, depth; preorder, inorder (binary trees), postorder, and level order; small decision/search trees	Draw a rooted tree from a small branching decision; trace each specified traversal correctly; interpret different visit orders.	Smaller CS5 — Exploring a Decision/Search Tree. Build a small branching model, follow a path to a decision, then compare traversal orders when exploring the whole tree. No claim that the tree learns from data.	Use a tiny nested node representation and print traversal order; explain new structure line by line. Animations belong to the later HTML module; manual traces come first.	Build a paper tree; learners act as visiting nodes; hand trace then Colab trace; brief tree-case reflection.
16	Integrated revision only; no new mathematical material: conceptual connections and final-examination preparation	Select an appropriate mathematical model for a familiar system and justify a solution; identify remaining misconceptions.	Compare rules, feature choices, route networks, and decision trees using the already taught mathematics.	Optional revisit of existing notebooks; no new syntax or packages.	Team mini-project showcase from earlier case studies; mixed-format mock questions and individual study plan.

